TEL200 Introduction to Robotics

Localization and Mapping

David A. Anisi

Ch. 6: Localization and Mapping

Background & pre-requisites

- Sailing vs. robotics
- Gaussian, White noise, PDF
- Estimation/Observer
  - Kalman Filter (KF/EKF)

Localization

- Dead reckoning (6.1)
- Localization with landmark/map (6.2)
- Particle Filter (PF) - Sequential Monte-Carlo Localization (6.6)

Mapping

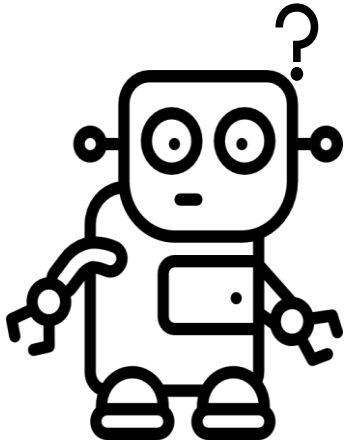
- Creating a map (6.3)

SLAM

- EKF SLAM (6.4)
- Rao-Blackwellized - FastSLAM (6.7)
- PoseGraph SLAM (6.5)

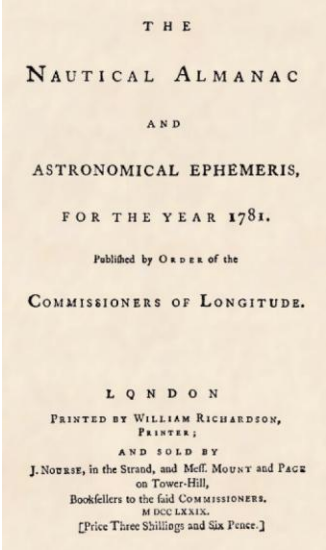
Robot localization

- Map-based planning requires localization- and map/mapping capability!
- Means of robot localization
  - GPS-like systems (GPS, ultrasound beacons, ...): useful when available, but not an option for many robotic applications, e.g., indoor, underwater, underground, and sometimes even outdoor (city, forests…)
  - Odometry: for instance, integrating the rotation speed of a wheel.
  - Using a map and observing landmarks.



Navigare necesse est.

- Classical robot localization methods such as dead reckoning and landmark-based navigation are nearly identical to maritime navigation (sailing)


THE

NAUTICAL ALMANAC

AND

ASTRONOMICAL EPHEMERIS,

FOR THE YEAR 1781.

Published by ORDER of the

COMMISSIONERS OF LONGITUDE.

LONDON

PRINTED BY WILLIAM RICHARDSON, PRINTER;

AND SOLD BY

J. NOURSE, in the Strand, and Messr. MOUNT and PAGE on Tower-Hill,

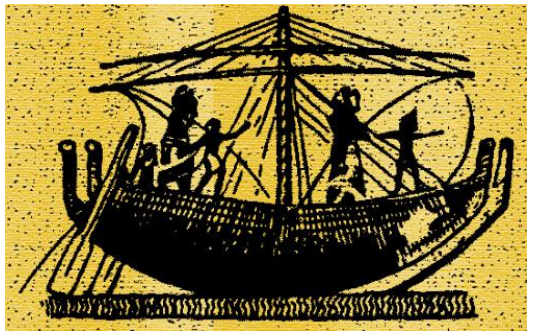
Booksellers to the said COMMISSIONERS.

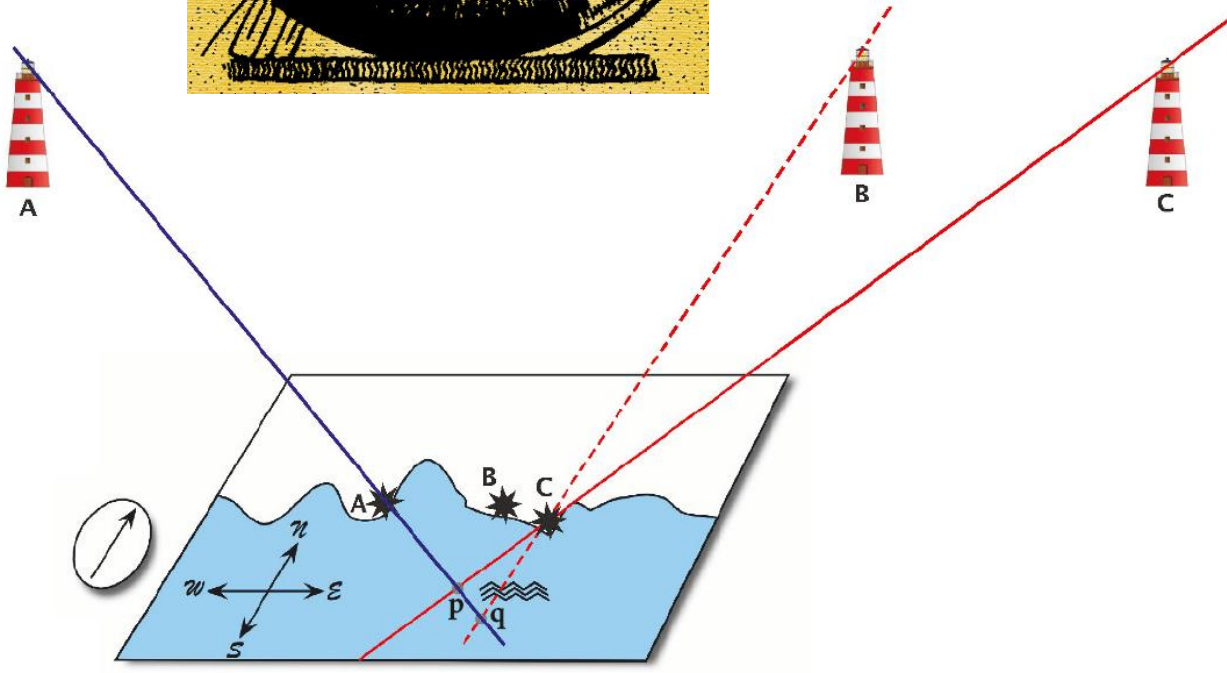
M DCC LXXIX.

[Price Three Shillings and Six Pence.]

Navigare necesse est.

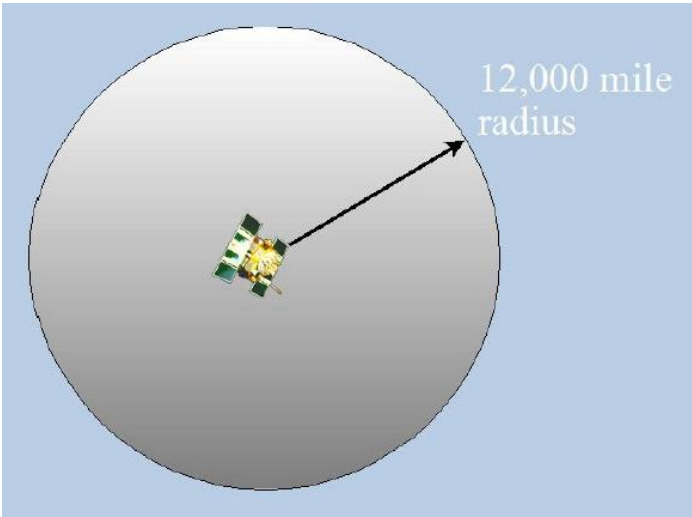
- Phoenicians navigated at sea 4000 years ago, 2000 years before compass was developed in China!
- Now using compass, map and landmarks: (flashing) lighthouses, buoys, stars, islands
- Resectioning (krysspeiling) for finding the position: 2+ bearings needed
- Other options: two leading lines (enslinje), one leading line and one bearing, two bearing on same object + sailed distance, bearing and distance, two distances, …



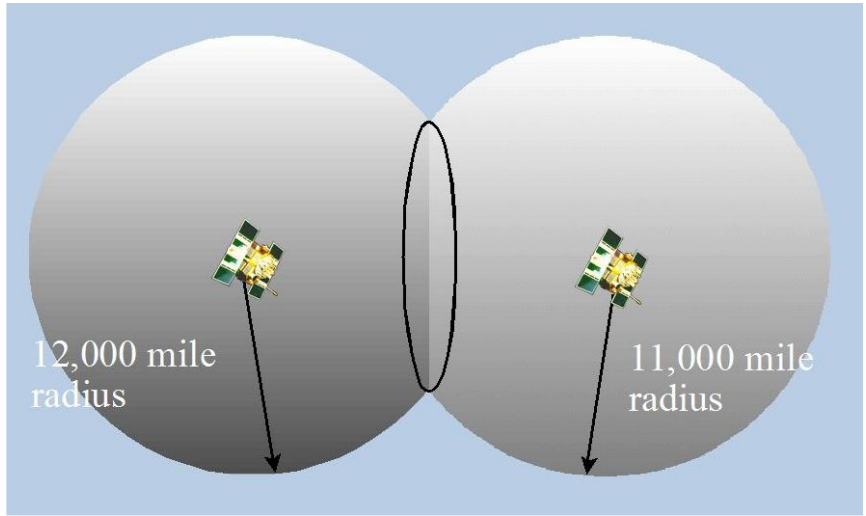


GPS: based on distance to 4+ known satellite positions

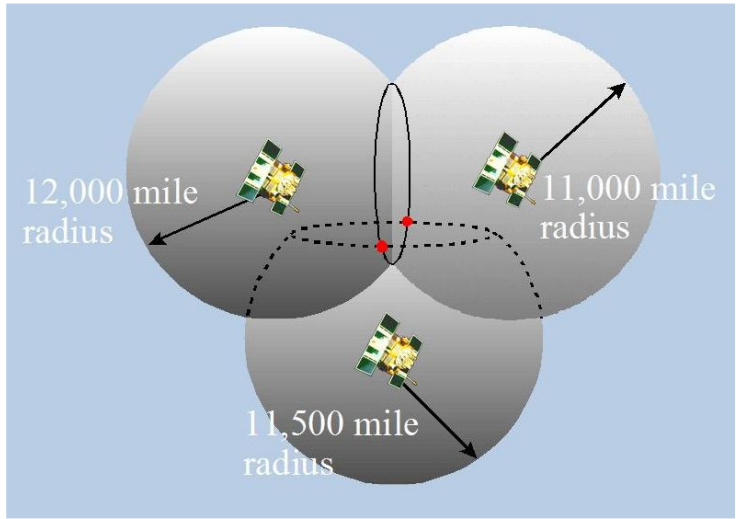
Distance to one satellite: sphere



Distance to two satellites: circle.



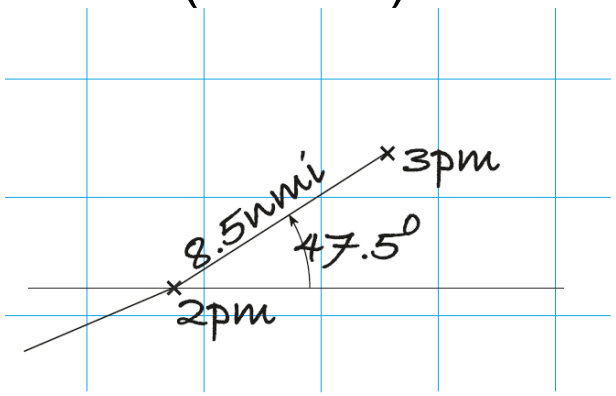
Distance to three satellites: point

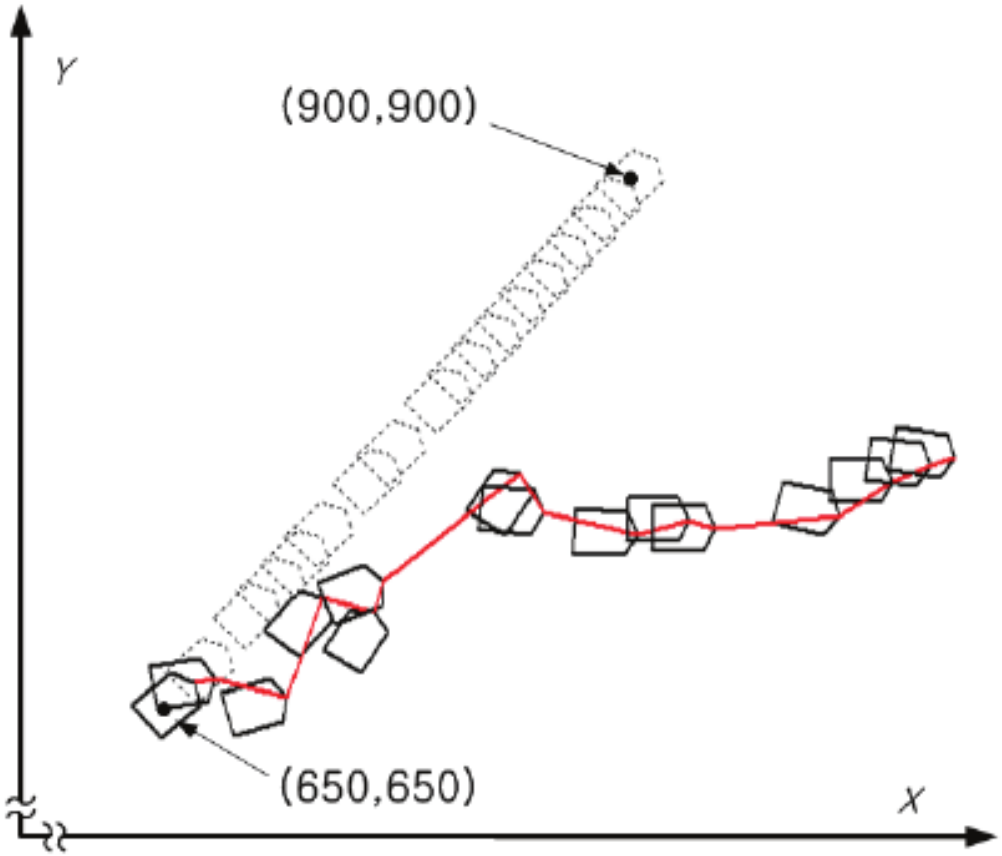


A fourth satellite is required for clock syncronization

Dead reckoning

- Estimating robot's pose based on its estimated speed, direction and time of travel with respect o previous estimate
- Recursive, error accumulation (unlimited)
- Corresponds to the prediction step of the Kalman Filter (Ch. 6.1)

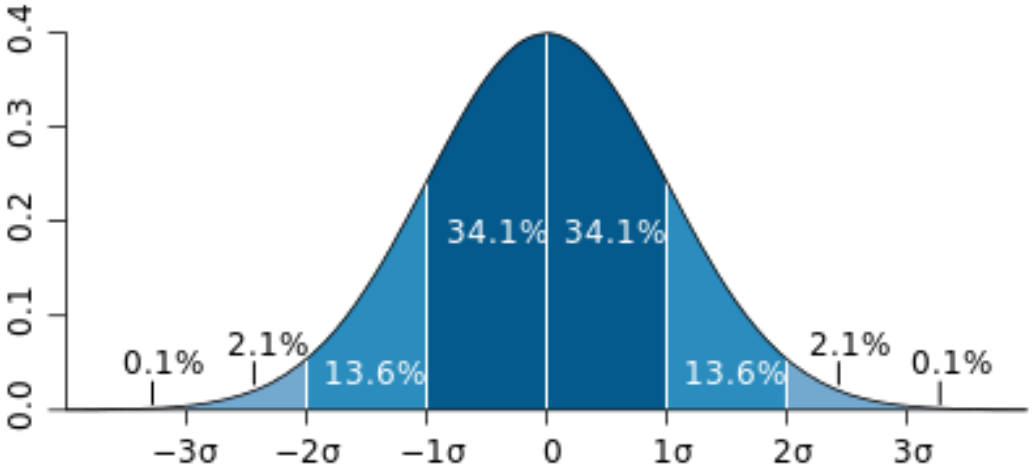


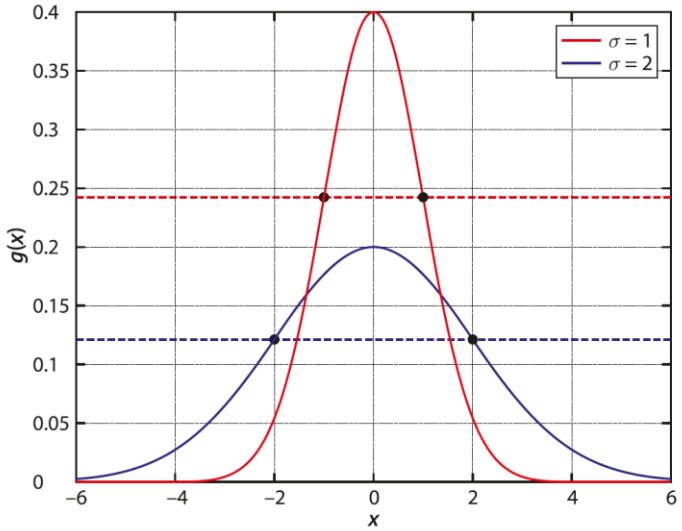


Bakgrunnsstoff: Gause-/normalfordeling

- Normalfordelt stokastisk variabel (Gaus-/bjellekurven)
  - Betegnelse: $N(\mu, \sigma)$
  - $g(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
- Sannsynlighet (probability)

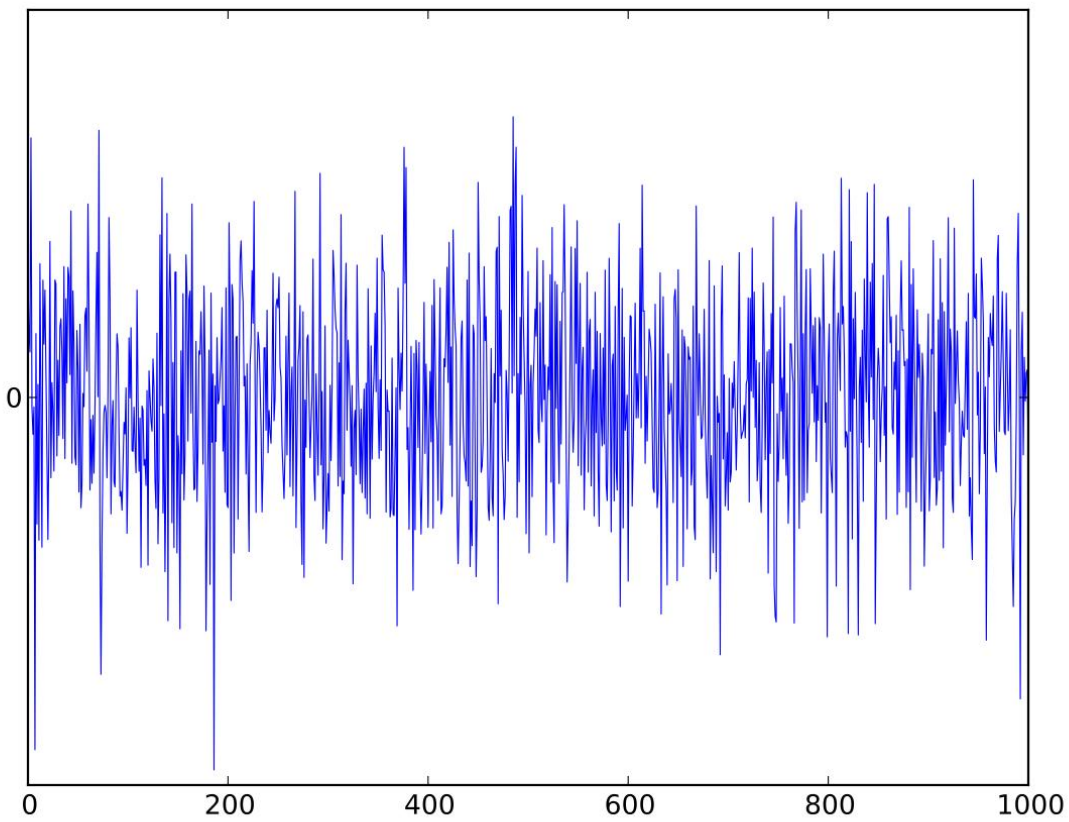
$$P(x \in [x_1, x_2]) = \int_{x_1}^{x_2} g(x)\mathrm{d}x$$





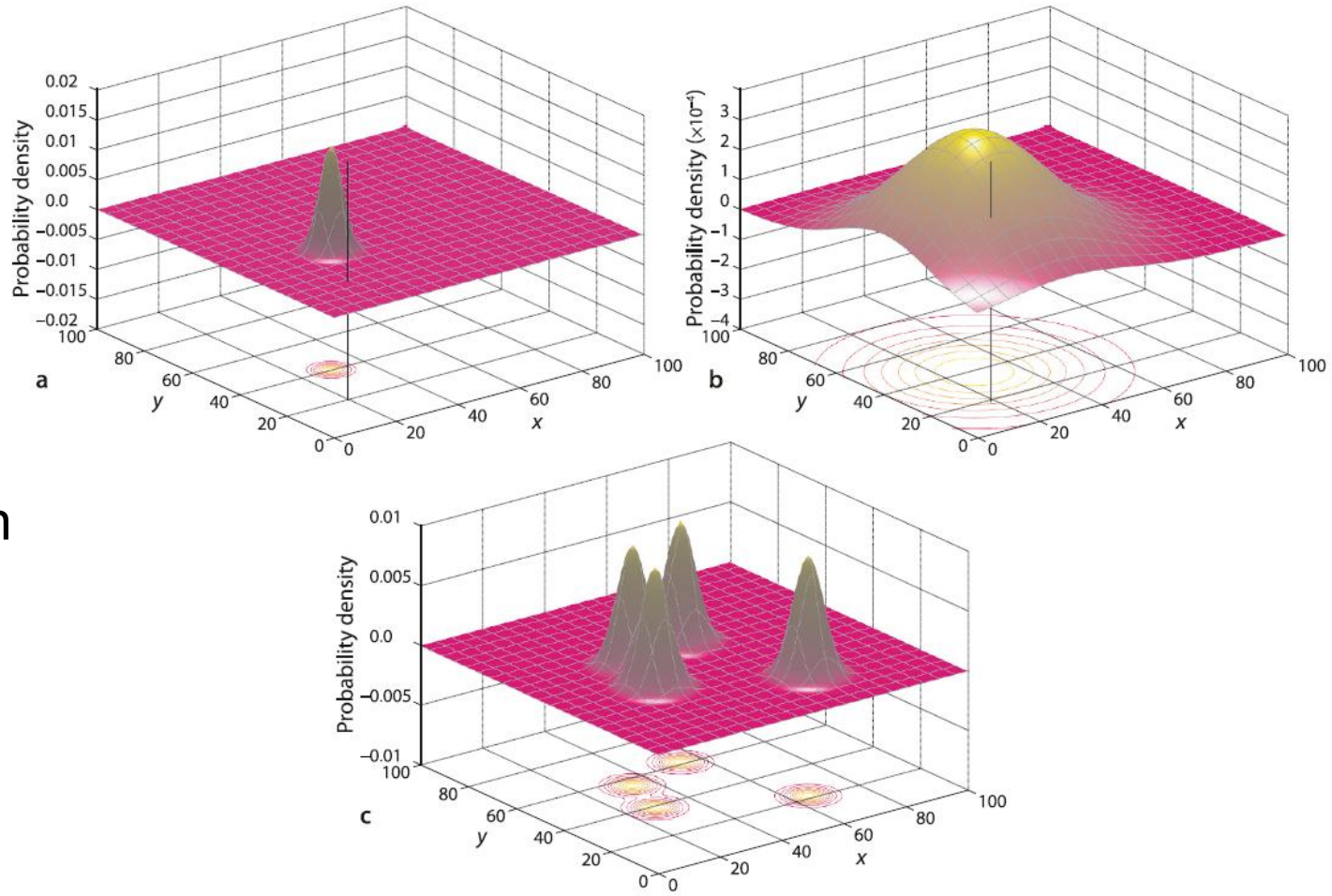
Bakgrunnsstoff: Hvit støy

- Hvit støy er en stokastisk signal som varierer helt tilfeldig og har konstant effekttetthet for alle frekvenser
  - Har null i middelverdi ($\mu = 0$)
  - Historisk verdi sier ingenting om neste/fremtidig verdi
  - Teoretisk kreves det ubegrenset energimengde for å generere



Bakgrunnstoff: sannsynlighetstetthetsfunksjon

- Probability Density Function (PDF)
- Probability = sanssynlighet
- Probability density = likelihood = sannsynlighetstetthet
- Volumen under PDF grafen i en liten region kring (x,y) gir sannsynligheten for at være i den regionen
- Area under PDF = 1
- PDF kan enkelt representere flere hypoteser



Student activity: position uncertainty, sketch the PDF on y-axis

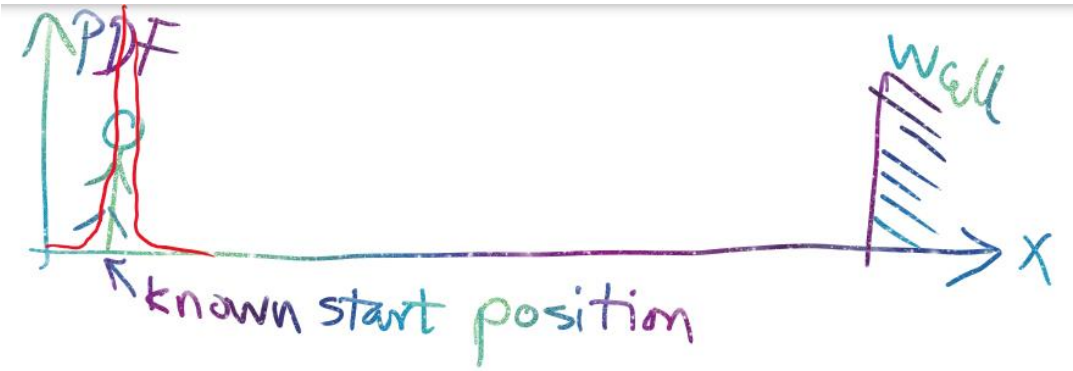
1. Person starts at known position (distance) from the wall

2. Person walks forward, counting steps and estimating motion.

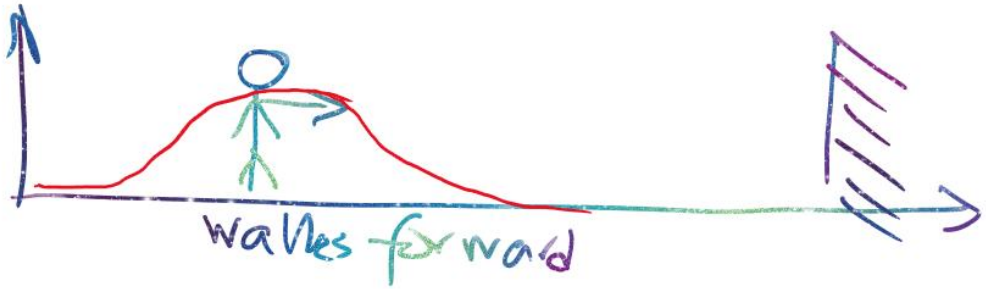
3. Person uses distance meter to get distance to wall.

Student activity: position uncertainty, sketch the PDF on y-axis

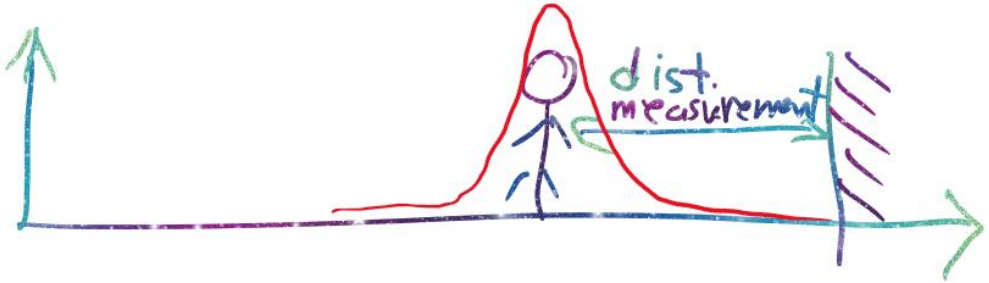
1. Person starts at known position (distance) from the wall



2. Person walks forward, counting steps and estimating motion. Uncertainty increases.



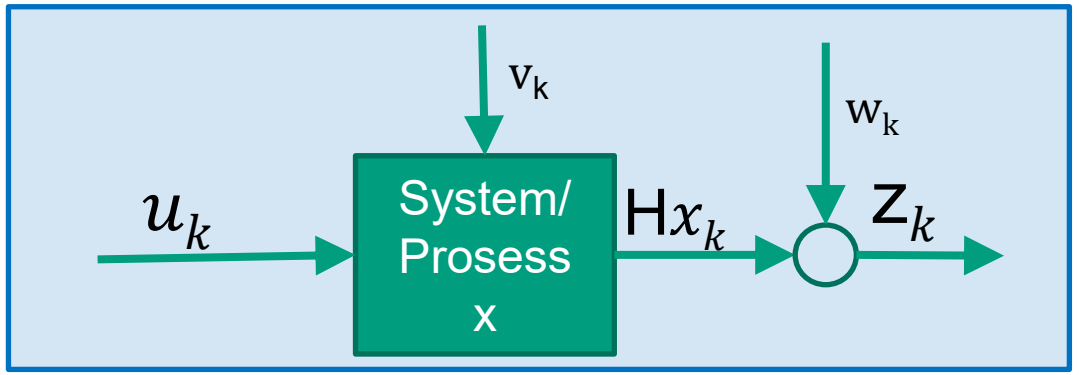
3. Person uses distance meter to get distance to wall. Uncertainty decreases. The more accurate measurement, the closer the updated position is to the measurement



Linjere dynamiske systemer (prosesser)

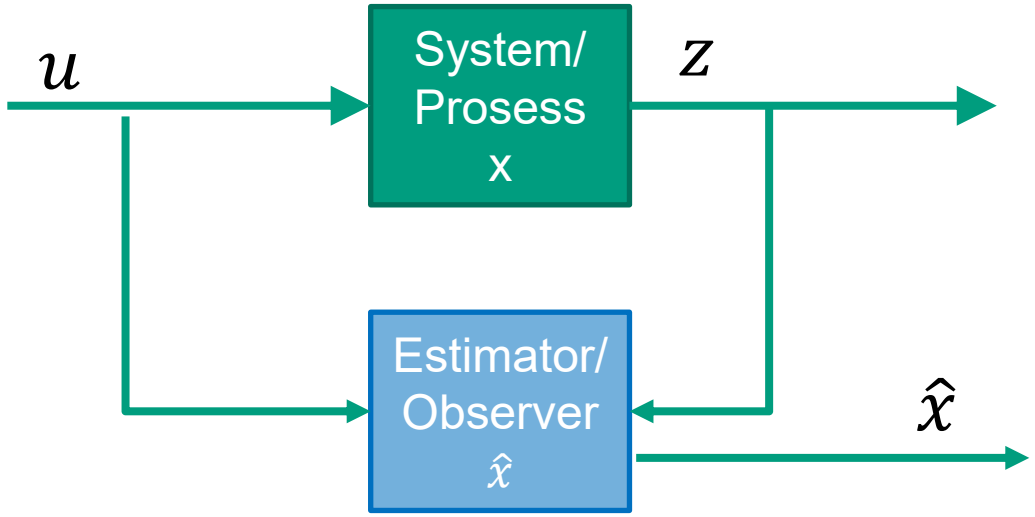
- Linjere dynamiske systemer
  - x_k = tilstand, tid k
  - u_k = input/pådragsvektor
  - z_k = output/målevektor
  - v_k = prosesstøy $\sim N(0, V)$
  - w_k = målestøy $\sim N(0, W)$
  - F,G,H matriser

$$x_{k+1} = F\, x_k + G\, uk + v_k$$
$$z_k = H\, x_k + w_k$$



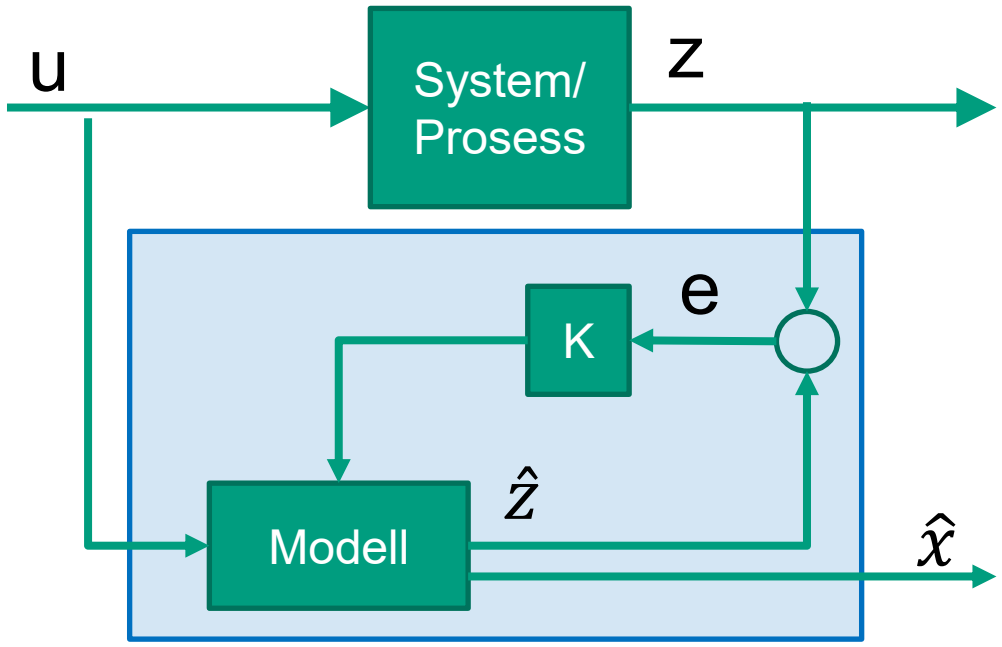
Tilstandsestimator

- En tilstandsestimator til en dynamisk system er en annen dynamisk system (matematisk modell) som kjører parallelt med den første og estimerer de tilstandsvariabler som vi ikke måler.
- Bruk av tilstandsestimator
  - Ikke fysisk målbart
  - Kostbar å måle
  - Unøyaktig eller støyfull måling
  - Feil på sensor



Hva er en Kalmanfilter?

- En rekursiv, modell-basert algoritme for optimal estimering av tilstandsvariabler av ett system med stokastiske forstyrelser og målestøy
- Optimal lineær filter: minimerer 2-normen av kovarians til estimeringsfeil, benytter all info
- Antagelser og merknader:
  - Systemet er linjert og observerbart
  - Trenger minst en måling
  - Prosess- og målestøy er hvit støy og uavhengig av hverandre



Kalmanfilter

Kalmanfilter algoritmen

Prediksjon

- Prediker (a priori) tilstandsestimat

$$\bar{x}_k = \mathrm{F}\hat{x}_{\mathrm{k-1}} + \mathrm{G}\, u_k$$

- Prediker estimatkovarians

$$\bar{P}_k = F\, \hat{P}_{k-1} F^T + \hat{V}$$

Korrigering

- Oppdater Kalmanfilter forsterkning

$$K_k = \bar{P}_k H^T \left(H \bar{P}_k H^T + \hat{W}\right)^{-1}$$

- Oppdater estimat med måling, z_k

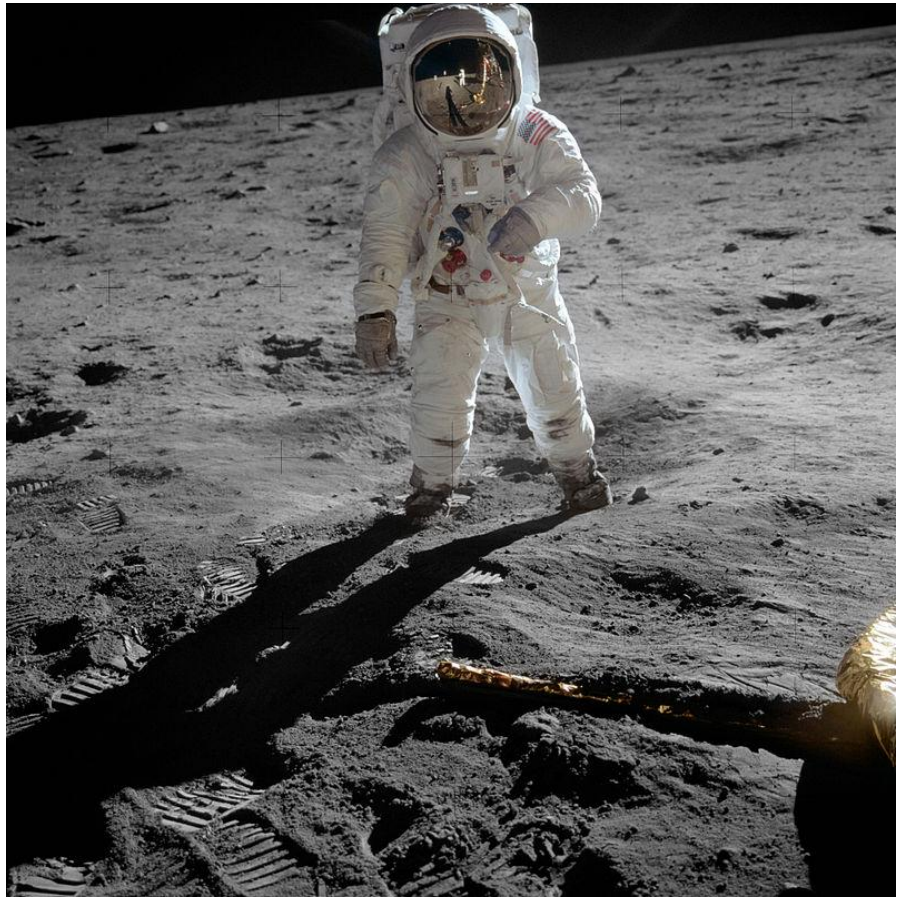
$$\hat{x}_k = \bar{x}_k + K\left(z_k - H\bar{x}_k\right)$$

- Oppdater estimatkovarians

$$\hat{P}_k = \left(I - K_k H\right) \bar{P}_k$$

Initialisering med $\hat{x}_0$ og $\hat{P}_0$

Hva bruker man Kalmanfilter til?



- Muliggjorde Apollo 11 månelanding, 1969
- Brukes nå i nesten all moderne apparater: fly, satellitter, mobiltelefoner, magnetisk resonansskanning, robot navigasjon, GPS posisjonering, auto-pilot for biler, værdata, aksjeprognoser, og …

Localization – 5min with Cyrill

https://www.youtube.com/watch?v=z92eUJjIJeY&list=PLgnQpQtFTOGSO8HC48K9sPuNliY1qxzV9
https://www.ipb.uni-bonn.de/5min/

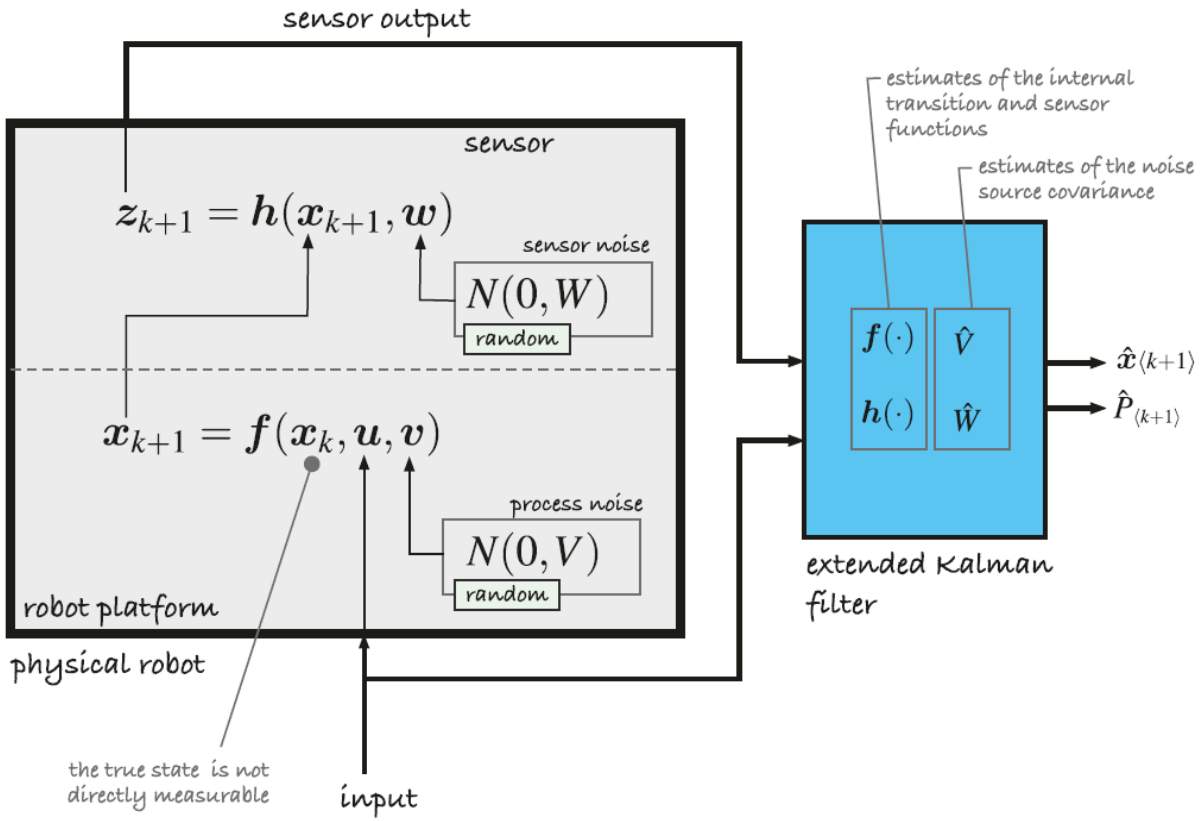
Kalman Filter (KF) – 5min with Cyrill

General (nonlinear) localization/estimation problem

Given: a model of the system f(·), h(·), $\hat{V}$ and $\hat{W}$;

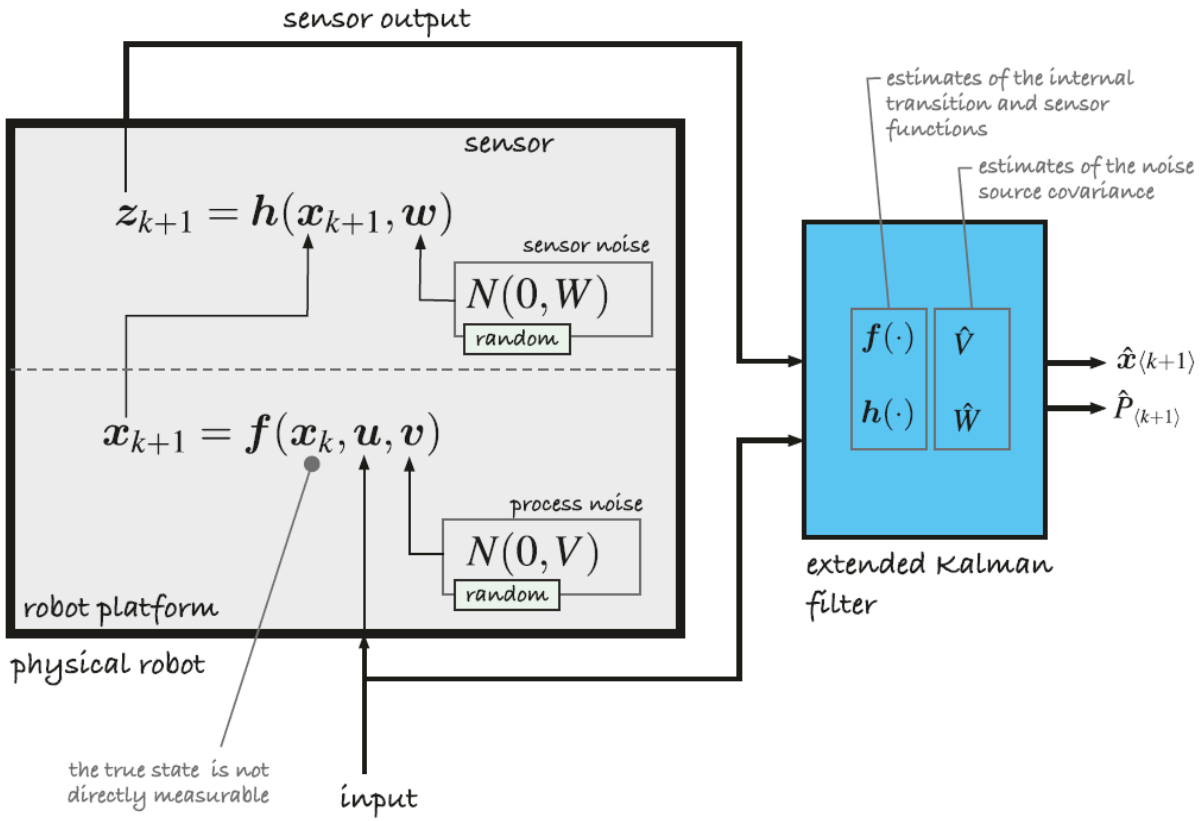
- known system inputs *u*; and
- some noisy sensor measurements, *z*

Find: an estimate $\hat{x}$ of the system state and our uncertainty $\hat{P}$ in that estimate.

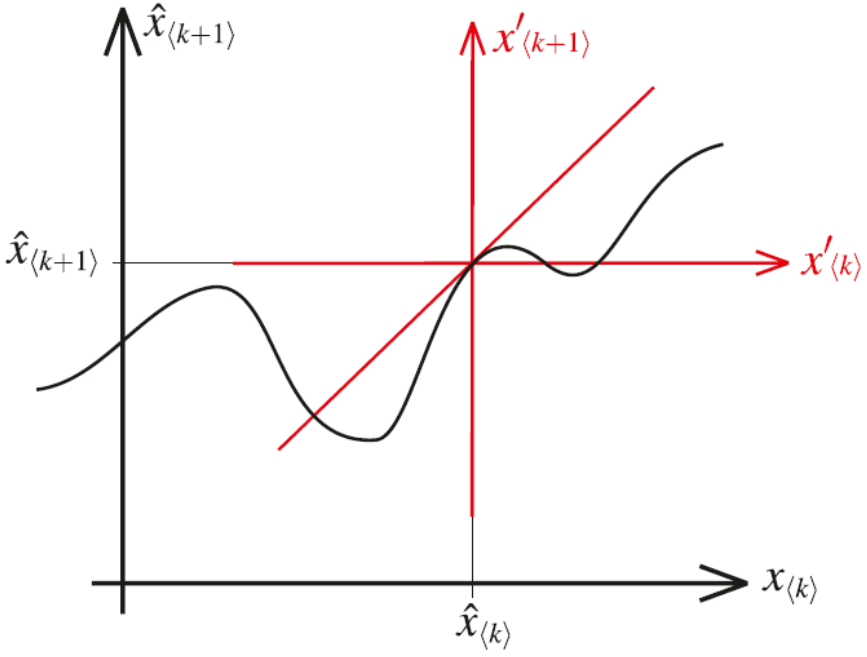


Localization in robotics

- x = true, unknown state/pose
- u = control commands
- z = output from various sensors
- $\hat{x}$ = our best prediction of x
- $\hat{P}$ = uncertainty in $\hat{x}$



Extended Kalman Filter (EKF) – for nonlinear systems



Procedure EKF

Input : $\hat{\boldsymbol{x}}\langle k\rangle \in \mathbb{R}^n$, $\hat{\boldsymbol{P}}\langle k\rangle \in \mathbb{R}^{n\times n}$, $\boldsymbol{u}\langle k\rangle \in \mathbb{R}^m$, $\boldsymbol{z}\langle k+1\rangle \in \mathbb{R}^p$, $\hat{\boldsymbol{V}} \in \mathbb{R}^{n\times n}$, $\hat{\boldsymbol{W}} \in \mathbb{R}^{p\times p}$

Output: $\hat{\boldsymbol{x}}\langle k+1\rangle \in \mathbb{R}^n$, $\hat{\boldsymbol{P}}\langle k+1\rangle \in \mathbb{R}^{n\times n}$

– *linearize about* $\boldsymbol{x} = \hat{\boldsymbol{x}}\langle k\rangle$

compute Jacobians: $\boldsymbol{F}_x \in \mathbb{R}^{n\times n}$, $\boldsymbol{F}_v \in \mathbb{R}^{n\times n}$, $\boldsymbol{H}_x \in \mathbb{R}^{p\times n}$, $\boldsymbol{H}_w \in \mathbb{R}^{p\times p}$

– *the prediction step*

$$\hat{\boldsymbol{x}}^+\langle k+1\rangle = \boldsymbol{f}\left(\hat{\boldsymbol{x}}\langle k\rangle, \boldsymbol{u}\langle k\rangle\right) \quad \text{// predict state at next time step}$$

$$\hat{\boldsymbol{P}}^+\langle k+1\rangle = \boldsymbol{F}_x\hat{\boldsymbol{P}}\langle k\rangle\boldsymbol{F}_x^T + \boldsymbol{F}_v\hat{\boldsymbol{V}}\boldsymbol{F}_v^T \quad \text{// predict covariance at next time step}$$

– *the update step*

$$\nu = \boldsymbol{z}\langle k+1\rangle - \boldsymbol{h}\left(\hat{\boldsymbol{x}}^+\langle k+1\rangle\right) \quad \text{// innovation : measured} - \text{predicted sensor value}$$

$$\boldsymbol{K} = \boldsymbol{P}^+\langle k+1\rangle\boldsymbol{H}_x^T\left[\boldsymbol{H}_x\boldsymbol{P}^+\langle k+1\rangle\boldsymbol{H}_x^T + \boldsymbol{H}_w\hat{\boldsymbol{W}}\boldsymbol{H}_w^T\right]^{-1} \quad \text{// Kalman gain}$$

$$\hat{\boldsymbol{x}}\langle k+1\rangle = \hat{\boldsymbol{x}}^+\langle k+1\rangle + \boldsymbol{K}\nu \quad \text{// update state estimate}$$

$$\hat{\boldsymbol{P}}\langle k+1\rangle = \hat{\boldsymbol{P}}^+\langle k+1\rangle - \boldsymbol{K}\boldsymbol{H}_x\hat{\boldsymbol{P}}^+\langle k+1\rangle \quad \text{// update covariance estimate}$$

Dead reckoning: bicycle model with odometry

- Odometer: sensor that measures the distance travelled by the robot, typically: wheel rotation angles.
- Keeping track of the direction of movement: differential odometry, compass, gyro.
- Affected by systematic error (wheel radius slightly off) and random errors (slippage).
- Bicycle vehicle model with odometry reading (distance/heading)

$$\dot{x} = v\cos\theta$$
$$\dot{y} = v\sin\theta$$
$$\dot{\theta} = \frac{v}{L}\tan\gamma$$

- System dynamics:

$$\boldsymbol{q}_{\langle k+1\rangle} = \boldsymbol{f}(\boldsymbol{q}_{\langle k\rangle},\ \boldsymbol{\delta}_{\langle k\rangle},\ \boldsymbol{v}_{\langle k\rangle}) = \begin{pmatrix} x_{\langle k\rangle} + (\delta_d + v_d)\cos\theta_{\langle k\rangle} \\ y_{\langle k\rangle} + (\delta_d + v_d)\sin\theta_{\langle k\rangle} \\ \theta_{\langle k\rangle} + \delta_\theta + v_\theta \end{pmatrix}$$

Dead reckoning: bicycle model with odometry

- Odometry reading: $\delta\langle k\rangle = (\delta_d, \delta_\theta)^T \in \mathbb{R}^{2\times 1}$
- Odometry covariance (process noice),**V.** Independent distance/heading errors.

$$\boldsymbol{v} = (v_d, v_\theta)^T \sim N(0, \boldsymbol{V}) \quad \boldsymbol{V} = \begin{pmatrix} \sigma_d^2 & 0 \\ 0 & \sigma_\theta^2 \end{pmatrix}$$

Python:

```
>>> V = np.diag([0.02, np.deg2rad(0.5)]) ** 2;
>>> robot = Bicycle(covar=V, animation="car")
>>> robot.control = RandomPath(workspace=10)
>>> robot.run(T=10);
```

Dead reckoning: bicycle model with odometry

- Prediction step of EKT requires P0 and Fx and Fv Jacobian matrices.

$$\boldsymbol{F}_x = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{v}=0} = \begin{pmatrix} 1 & 0 & -\delta_d \sin\theta_v \\ 0 & 1 & \delta_d \cos\theta_v \\ 0 & 0 & 1 \end{pmatrix} \quad \boldsymbol{F}_v = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{v}}\right|_{\boldsymbol{v}=0} = \begin{pmatrix} \cos\theta_v & 0 \\ \sin\theta_v & 0 \\ 0 & 1 \end{pmatrix}$$

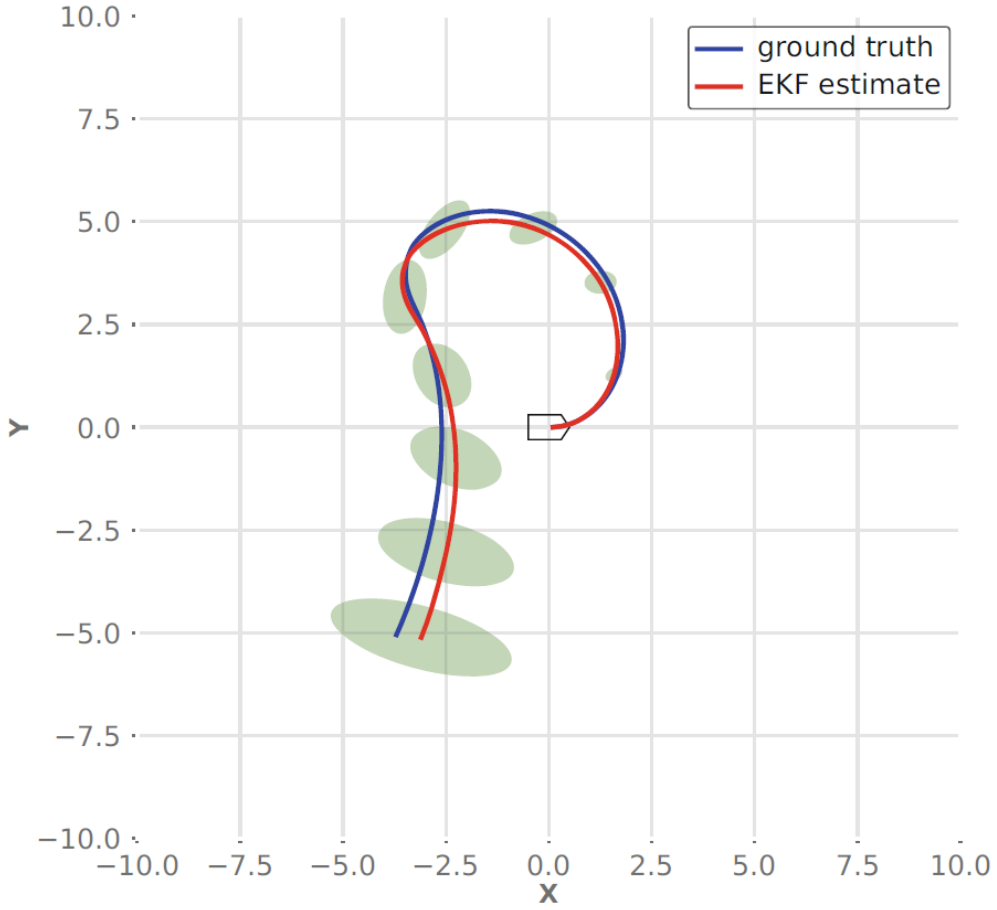
- Bicycle vehicle object contains methods for calculating Fx and Fv (veh.Fx, veh.Fv)

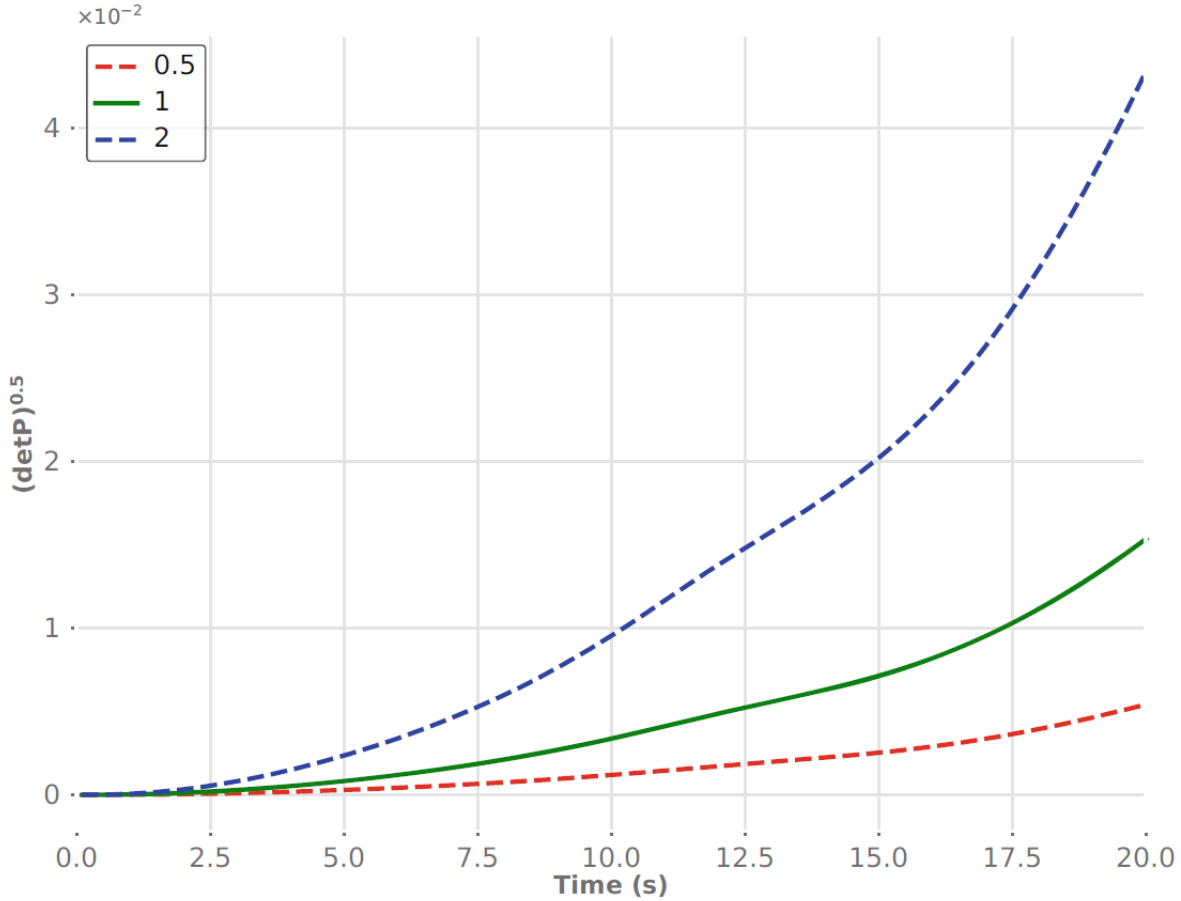
Python:

```
>>> P0 = np.diag([0.05, 0.05, np.deg2rad(0.5)]) ** 2;
>>> ekf = EKF(robot=(robot, V), P0=P0)
>>> ekf.run(T=20);
```

Dead reckoning: bicycle model with odometry

```
>>> robot.plot_xy(color="b"); ekf.plot_xy(color="r");
```





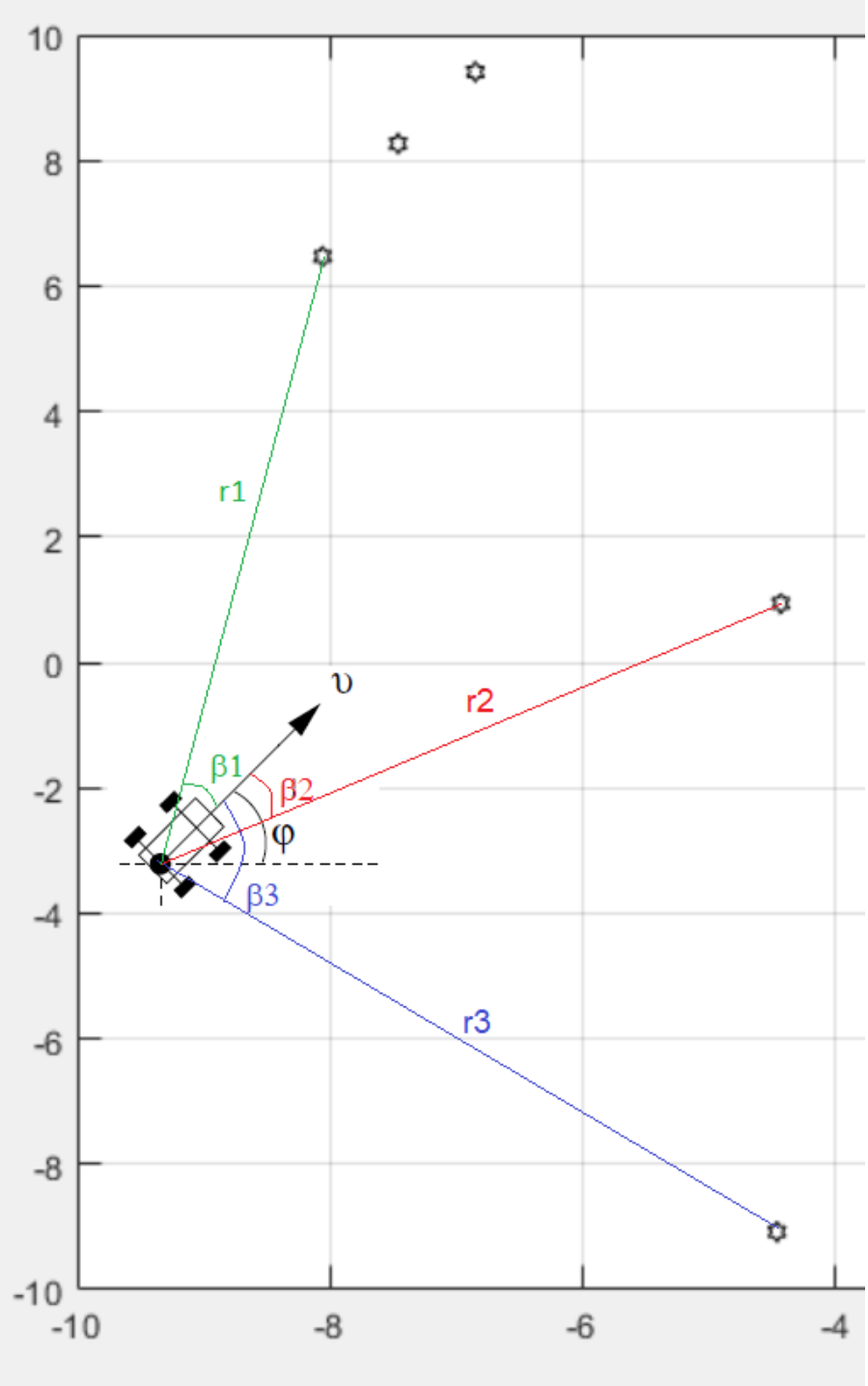
EKF localization with map & landmarks

- Bicycle vehicle model with odometry covariance (process noice),**V.** Independent distance/heading errors.

$$\dot{x} = v\cos\theta$$
$$\dot{y} = v\sin\theta$$
$$\dot{\theta} = \frac{v}{L}\tan\gamma$$

$$\boldsymbol{V} = \begin{pmatrix} \sigma_d^2 & 0 \\ 0 & \sigma_\theta^2 \end{pmatrix}$$

```
>>> map = LandmarkMap(20, workspace=10)
>>> V = np.diag([0.02, np.deg2rad(0.5)]) ** 2
>>> robot = Bicycle(covar=V, animation="car");
>>> robot.control = RandomPath(workspace=map)
```



Localization with map and landmarks

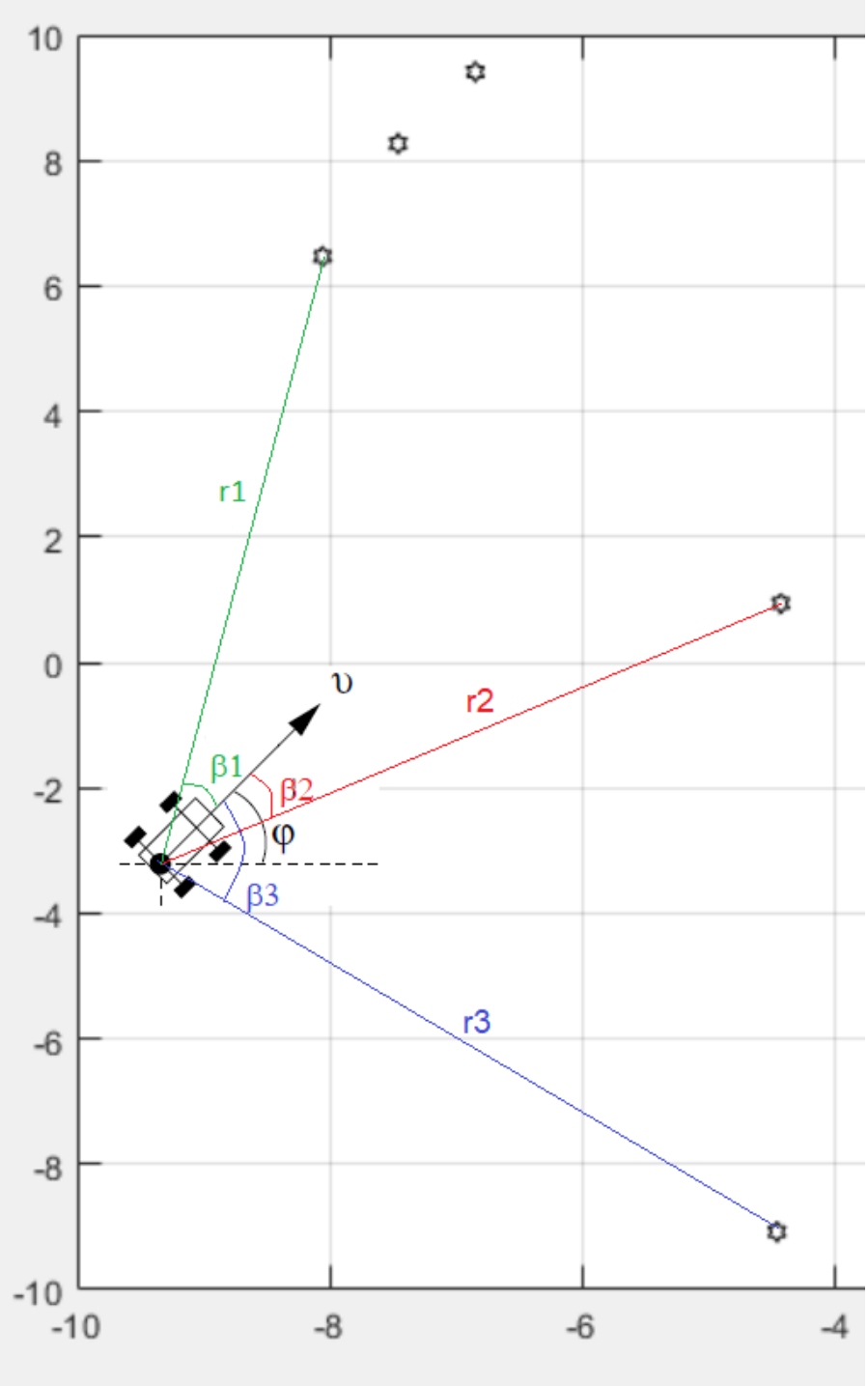
- Robot equipped with **RangeBearingSensor** measuring range and bearing to landmark i: $(r_i, \beta_i)^T$

$$z = h(q,\ p_i) = \begin{pmatrix} \sqrt{(y_i - y_v)^2 + (x_i - x_v)^2} \\ \tan^{-1} \frac{y_i - y_v}{x_i - x_v} - \theta_v \end{pmatrix} + \begin{pmatrix} w_r \\ w_\beta \end{pmatrix}$$

- Measurement noise $w = (w_r, w_\beta)^T \sim N(0, \boldsymbol{W})$

$$\boldsymbol{W} = \begin{pmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\beta^2 \end{pmatrix}$$

```
>>> W = np.diag([0.1, np.deg2rad(1)]) ** 2;
>>> sensor = RangeBearingSensor(robot=robot, map=map, covar=W,. angle=[-pi/2, pi/2], range=4, animate=True)
```

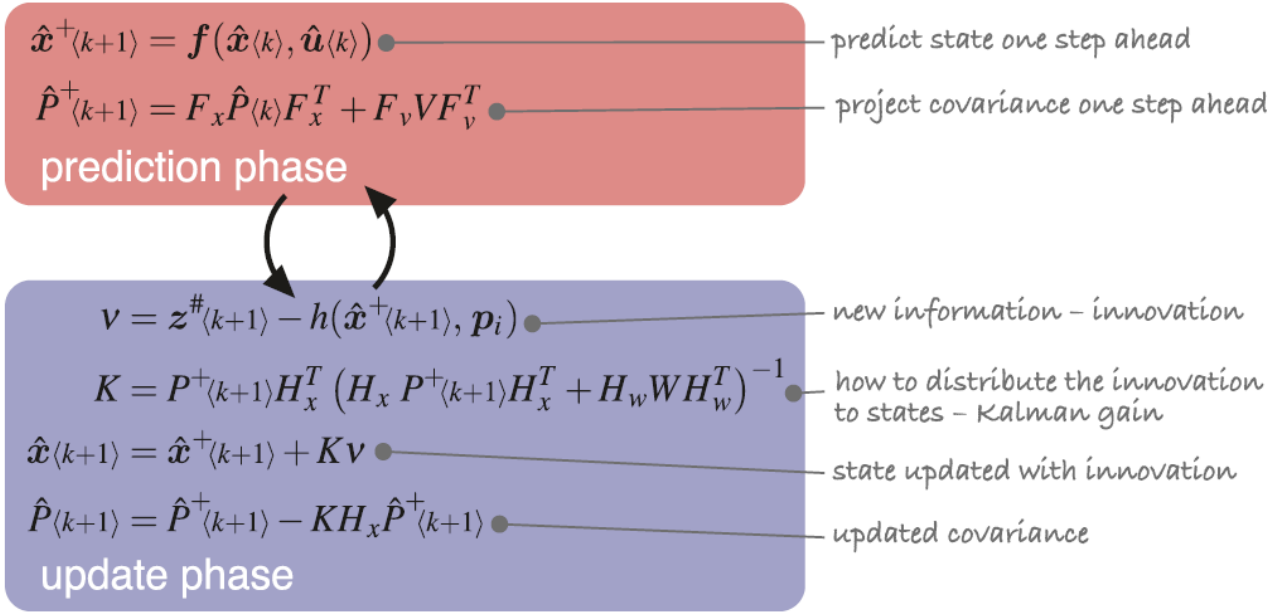


Localization with map and landmarks

- The RangeBearingSensor object includes methods h to implement sensor reading and Hx and Hw to compute the Jacobians.

$$H_x = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}\right|_{w=0} = \begin{pmatrix} -\frac{x_i - x_v}{r} & -\frac{y_i - y_v}{r} & 0 \\ \frac{y_i - y_v}{r^2} & -\frac{x_i - x_v}{r^2} & -1 \end{pmatrix}$$

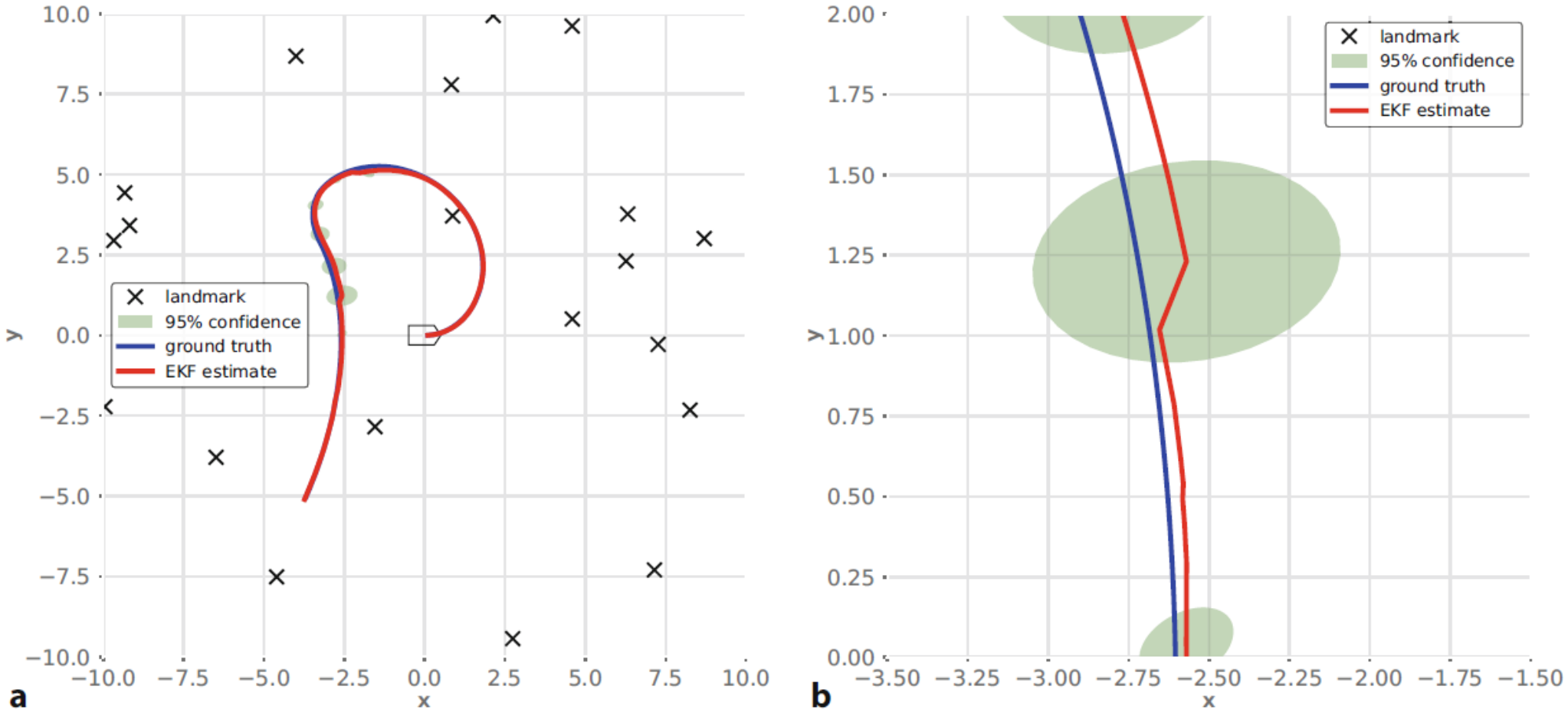
$$H_w = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{w}}\right|_{w=0} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$




```
>>> P0 = np.diag([0.05, 0.05, np.deg2rad(0.5)]) ** 2;
>>> ekf = EKF(robot=(robot, V), P0=P0, map=map, sensor=(sensor, W));
```

Localization with map and landmarks

```
>>> ekf.run(T=20); map.plot(); robot.plot_xy(); ekf.plot_xy(); ekf.plot_ellipse()
```



Localization with map and landmarks

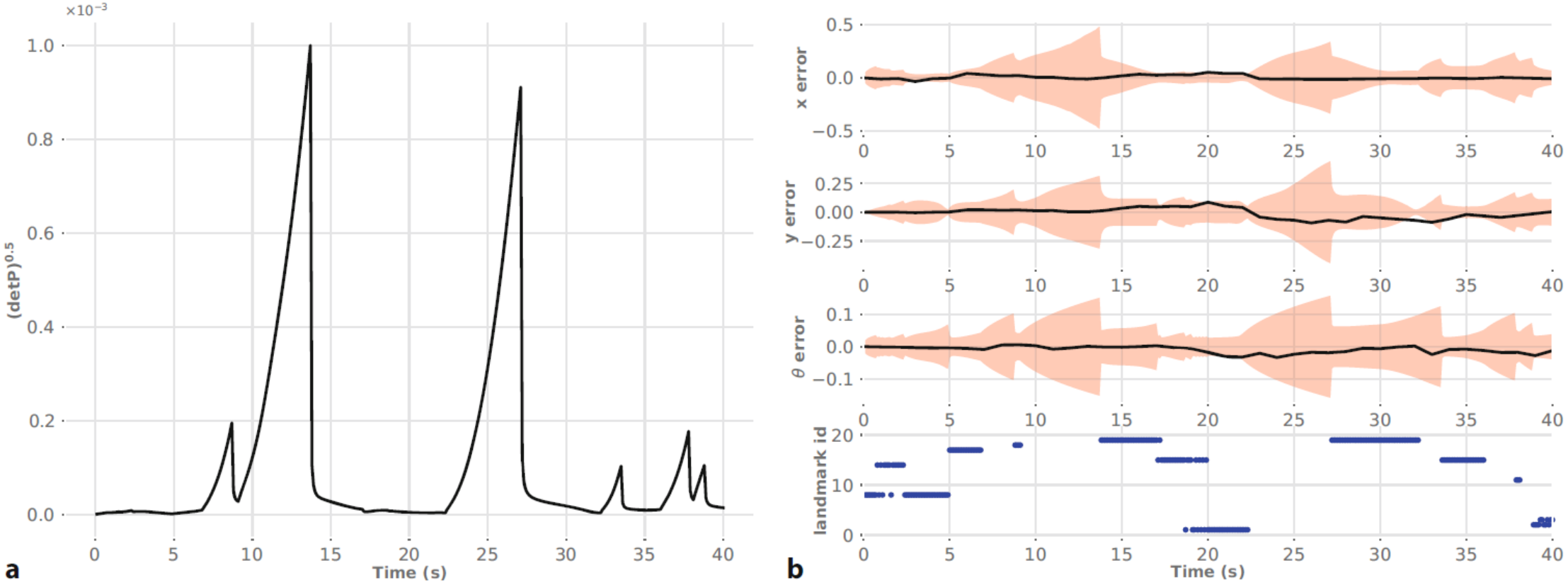


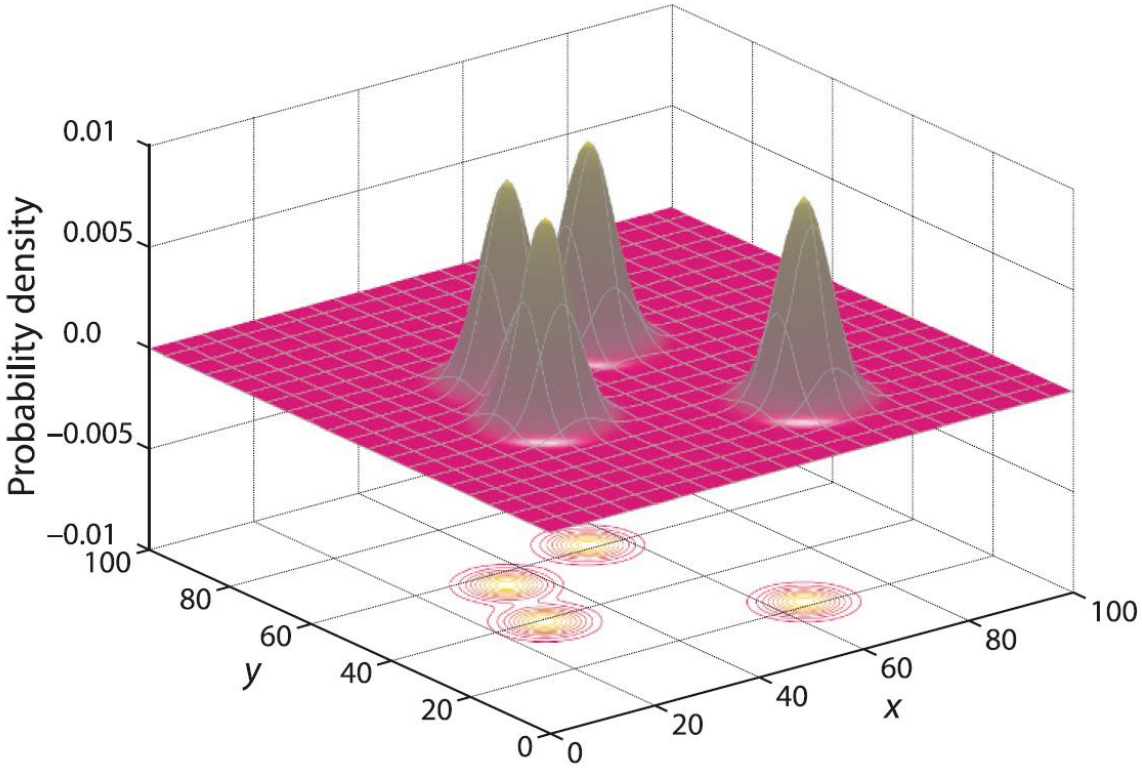
Fig. 6.10 **a** Covariance magnitude as a function of time. Overall uncertainty is given by $\sqrt{\det(\mathbf{P})}$ and shows that uncertainty does not continually increase with time. **b** Top: pose estimation error with 95% confidence bound shown in coral; bottom: scatter plot of observed landmarks and periods of no observations can be clearly seen

Particle Filter (PF) - Sequential Monte-Carlo Localization

<u>Assumptions/limitations of EKF:</u>

- Limited to linear/linearized, observable systems
- Risk of quick divergence if the initial conditions or dynamic/measurement models are incorrect.
- Gaussian (white) and independent PDF
- Single-peak (uni-modal) uncertainty distribution function

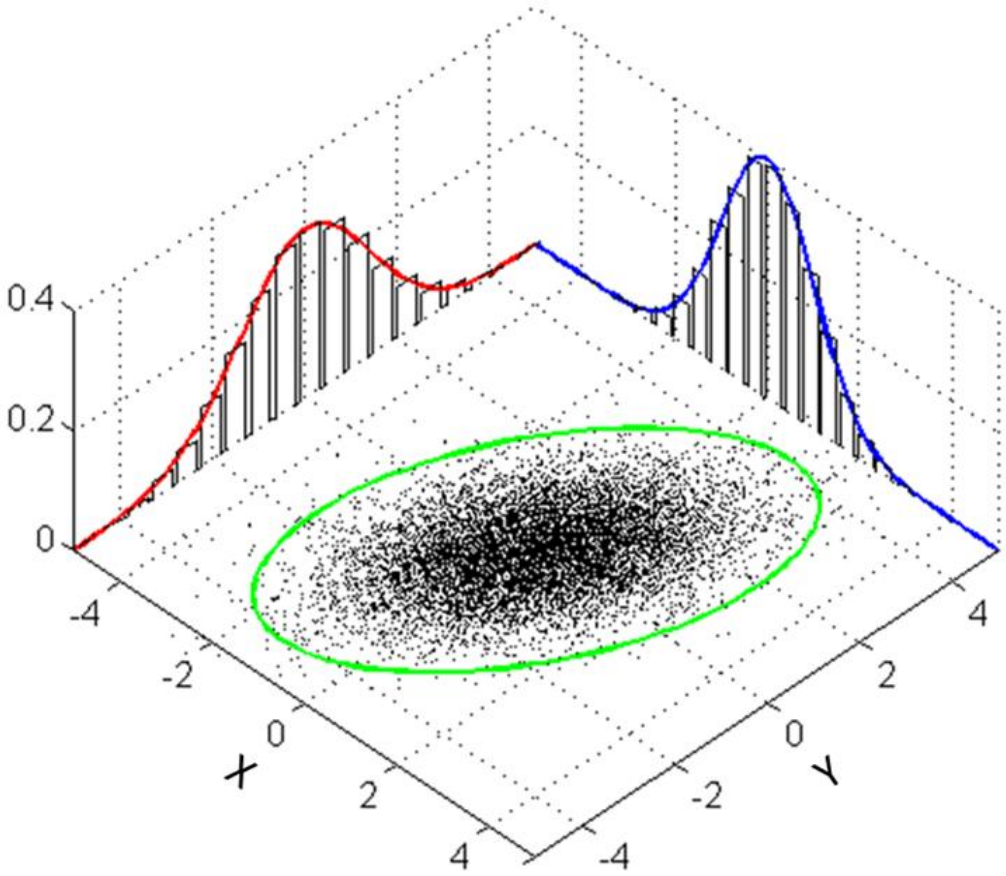
Particle filter circumvent all these issues!



Particle Filter (PF) - Sequential Monte-Carlo Localization

Particle filter/Monte-Carlo sampling

- Don’t care about model linearity
- Handles non-gaussian representation and uncertainty
- Represent multi-modal hypothesis hence circumvent data association problem
- But is computationally much more expensive.
- Potential for GPU parallelization..



Particle Filter (PF) - Sequential Monte-Carlo Localization

Initialization: Maintain N states concurrently, with weight $w_i = 1/N$

1) Prediction phase: Randomly perturb states when updating them with the dynamics model

2) Update/re-sampling: Keep N, and get rid of remaining particles that do not explain new measurements well

- Pose estimate: mean of all particles.
- Pose uncertainty: standard deviation

prediction phase

$$\boldsymbol{x}_i^{+}{}_{\langle k+1\rangle} = \boldsymbol{f}(\boldsymbol{x}_{i\langle k\rangle}, \boldsymbol{u}_{\langle k\rangle} + \boldsymbol{q}_{\langle k\rangle})$$

predict state of this particle one step ahead based on inputs and model

add a random value to represent uncertainty

update phase

$$\boldsymbol{v}_i = \boldsymbol{z}^{\#}{}_{\langle k+1\rangle} - h(\hat{\boldsymbol{x}}_i^{+}{}_{\langle k+1\rangle}, \boldsymbol{p}_j)$$

new information – innovation – from observing landmark j

$$w_i = \mathrm{e}^{-\boldsymbol{v}_i^T L^{-1} \boldsymbol{v}_i} + w_0$$

weight is a function of how likely the measurement was

$$w_i' = w_i / \sum_{i=1}^{N} w_i$$

normalize the weights 0 to 1

$$\boldsymbol{x}_{\langle k+1\rangle} \leftarrow R(\boldsymbol{x}^{+}{}_{\langle k+1\rangle}, \boldsymbol{w})$$

resample, weighted selection of particles to go to the next round

Particle Filter (PF) - Sequential Monte-Carlo Localization

```
>>> map = LandmarkMap(20, workspace=10);
>>> V = np.diag([0.02, np.deg2rad(0.5)]) ** 2;
>>> robot = Bicycle(covar=V, animation="car",
workspace=map);
>>> robot.control = RandomPath(workspace=map)
>>> W = np.diag([0.1, np.deg2rad(1)]) ** 2;
>>> sensor = RangeBearingSensor(robot, map,
covar=W, plot=True);
```

Covariance applied to state dynamics:

```
>>> Q = np.diag([0.1, 0.1, np.deg2rad(1)]) ** 2;
```

Covariance applied to innovation:

```
>>> L = np.diag([0.1, 0.1]);
```

prediction phase

$$\boldsymbol{x}_i^{+}{}_{\langle k+1\rangle} = \boldsymbol{f}(\boldsymbol{x}_i{}_{\langle k\rangle}, \boldsymbol{u}_{\langle k\rangle} + \boldsymbol{q}_{\langle k\rangle})$$

predict state of this particle one step ahead based on inputs and model

add a random value to represent uncertainty

update phase

$$\boldsymbol{v}_i = \boldsymbol{z}^{\#}{}_{\langle k+1\rangle} - \boldsymbol{h}(\hat{\boldsymbol{x}}_i^{+}{}_{\langle k+1\rangle}, \boldsymbol{p}_j)$$

new information – innovation – from observing landmark j

$$w_i = \mathrm{e}^{-\boldsymbol{v}_i^T L^{-1} \boldsymbol{v}_i} + w_0$$

weight is a function of how likely the measurement was

$$w_i' = w_i / \sum_{i=1}^{N} w_i$$

normalize the weights 0 to 1

$$\boldsymbol{x}_{\langle k+1\rangle} \leftarrow R(\boldsymbol{x}^{+}{}_{\langle k+1\rangle}, \boldsymbol{w})$$

resample, weighted selection of particles to go to the next round

Particle Filter (PF) - Sequential Monte-Carlo Localization

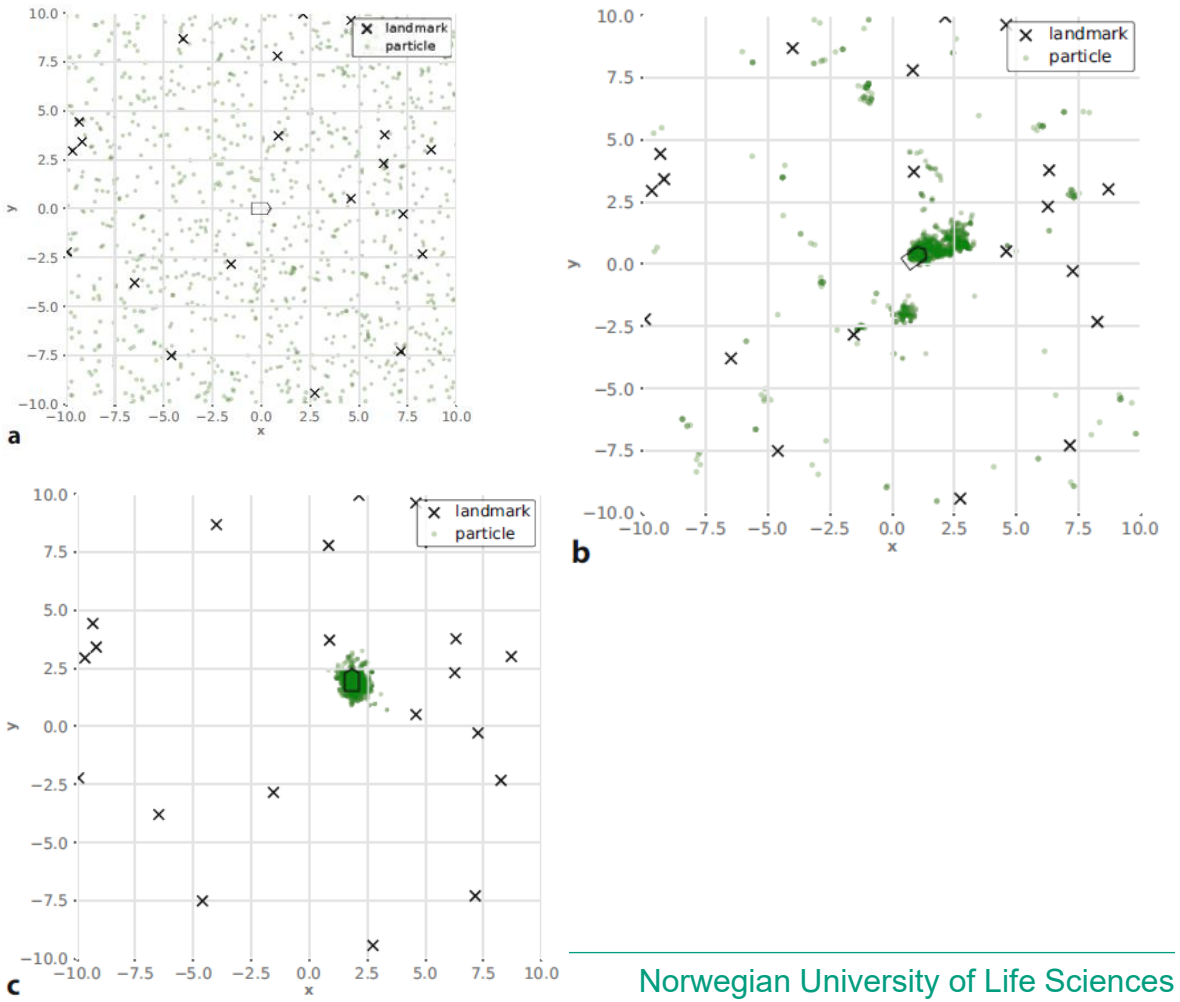
```
>>> map = LandmarkMap(20, workspace=10);
>>> V = np.diag([0.02, np.deg2rad(0.5)]) ** 2;
>>> robot = Bicycle(covar=V, animation="car",
workspace=map);
>>> robot.control = RandomPath(workspace=map)
>>> W = np.diag([0.1, np.deg2rad(1)]) ** 2;
>>> sensor = RangeBearingSensor(robot, map,
covar=W, plot=True);
```

Covariance applied to state dynamics:

```
>>> Q = np.diag([0.1, 0.1, np.deg2rad(1)]) ** 2;
```

Covariance applied to innovation:

```
>>> L = np.diag([0.1, 0.1]);
```



Particle Filter (PF) - Sequential Monte-Carlo Localization

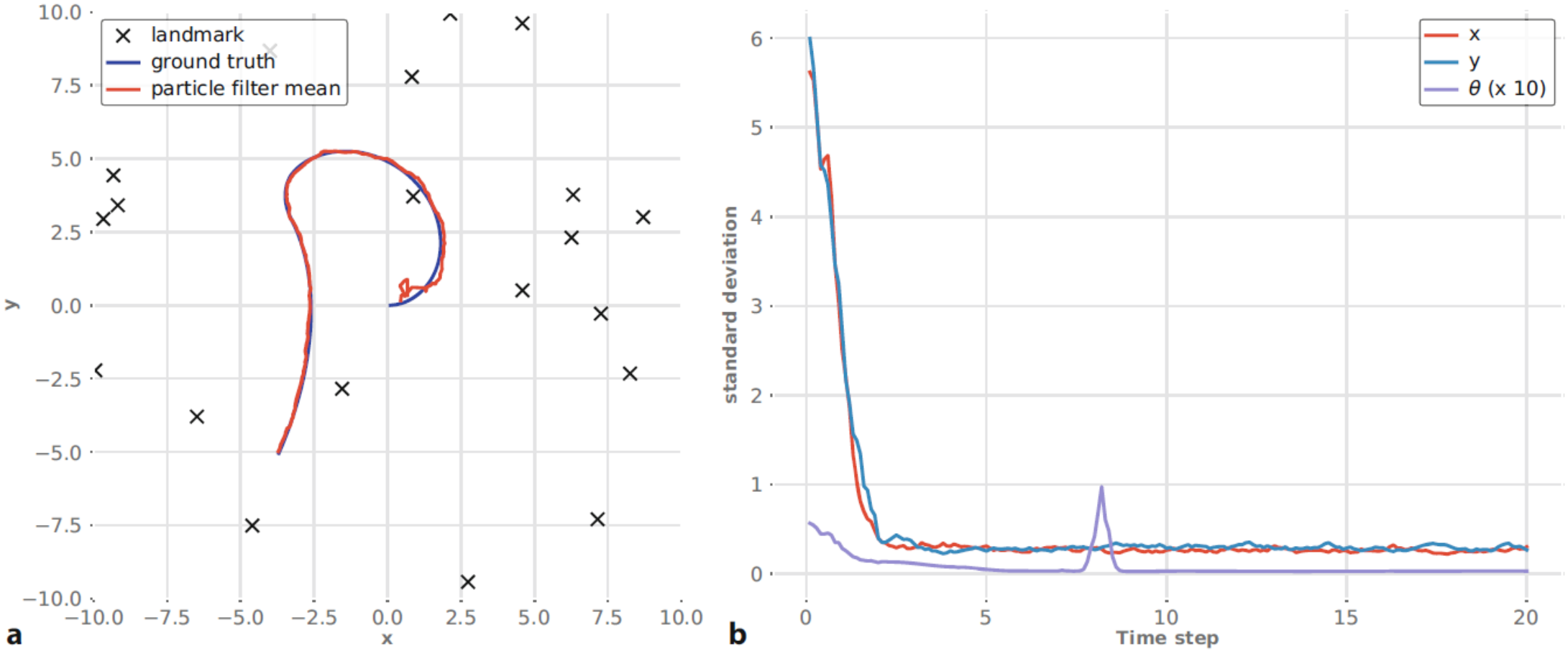


Fig. 6.24 Particle filter results. **a** Robot path; **b** standard deviation of the particles versus time

Particle Filter (PF) – 5min with Cyrill

Thank you for your attention!